

MARIE-ANDRÉE HEALEY-CÔTÉ

Cadriciel Web

582-41B-MA gr.24610

EXERCICE #1: LARAVEL

Travail présenté à

M. Marcos Sanches

Département de Formation continue – Programme NWE.OF

Collège de Maisonneuve

Le 27 janvier 2026

Vous devez créer un site web statique avec Laravel, cela peut être la présentation d'un produit, un résumé, etc.

Voici les instructions réécrites pour l'exercice :

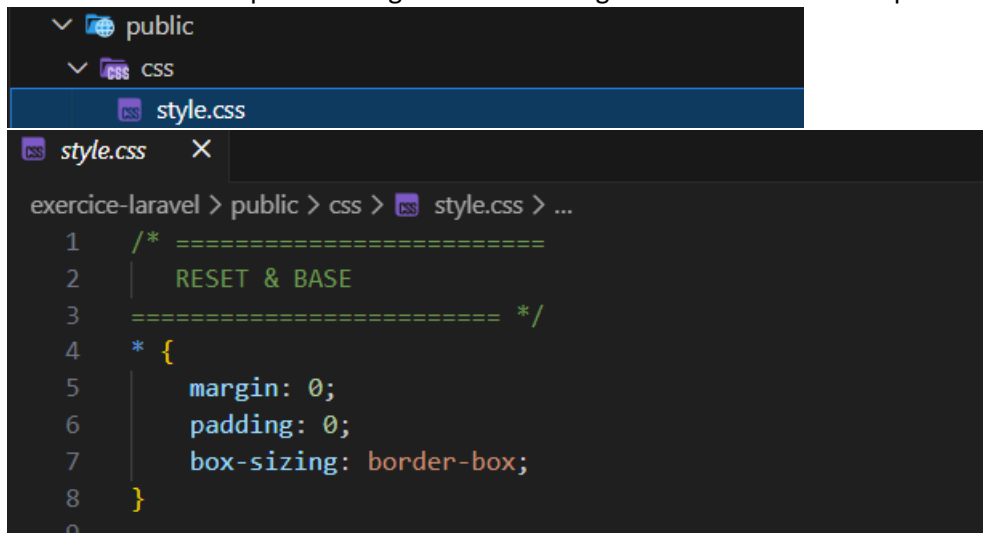
1. Création du projet Laravel : (2pts)

- Utilisez la ligne de commande pour créer un nouveau projet Laravel.

```
amy14@MSI MINGW64 ~/Documents/mesProjets/cadricielWeb  
$ composer create-project --prefer-dist laravel/laravel exercice-laravel
```

2. Création du fichier CSS : (2pts)

- Créez un fichier CSS pour le design du site et enregistrez-le dans le dossier public.



The screenshot shows a file explorer on the left with the following structure:

- public
 - css
 - style.css

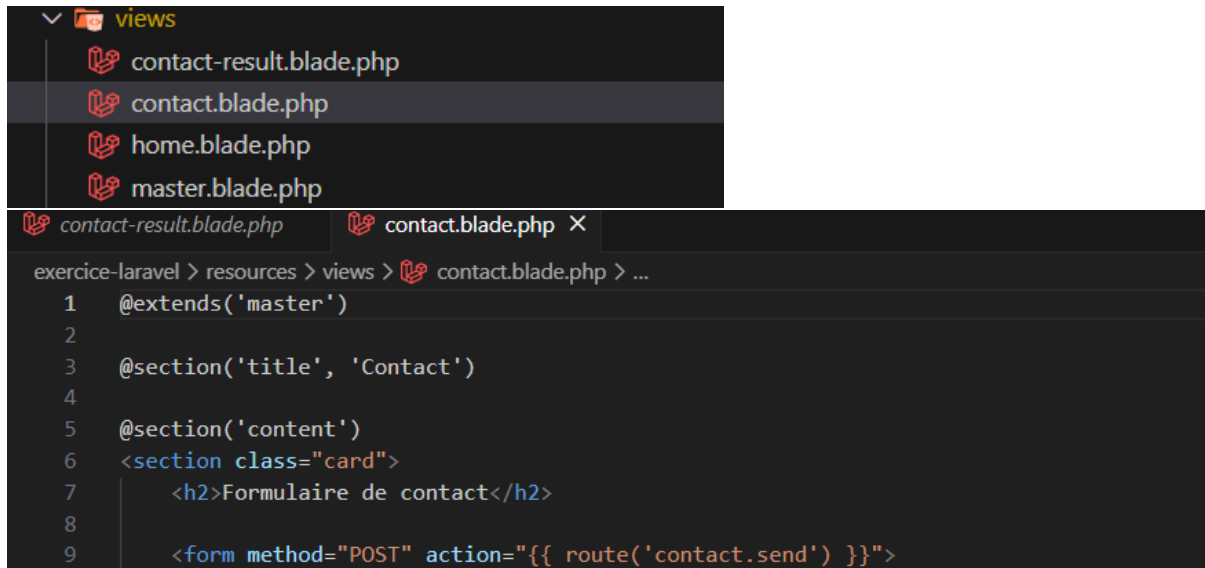
The main editor window shows the content of `style.css` with the following code:

```
exercice-laravel > public > css > style.css > ...  
1  /* =====  
2  |  RESET & BASE  
3  |  ===== */  
4  * {  
5  |  margin: 0;  
6  |  padding: 0;  
7  |  box-sizing: border-box;  
8  }  
9
```

- Vous pouvez utiliser un thème existant, mais assurez-vous d'inclure le lien et les références dans la documentation.

3. Ajuster le contenu selon vos données :

- Si vous avez utilisé des « templates », vous devez ajuster leur contenu avec vos propres données. Les « templates » sans modification du contenu ne seront pas acceptés et entraîneront l'attribution d'une note zéro.



The screenshot shows a code editor with a file explorer on the left displaying the 'views' directory. The files listed are 'contact-result.blade.php', 'contact.blade.php' (selected), 'home.blade.php', and 'master.blade.php'. The main editor area shows the content of 'contact.blade.php' with the following code:

```
exercice-laravel > resources > views > contact.blade.php > ...
1  @extends('master')
2
3  @section('title', 'Contact')
4
5  @section('content')
6      <section class="card">
7          <h2>Formulaire de contact</h2>
8
9          <form method="POST" action="{{ route('contact.send') }}">
```

4. Création du contrôleur : (2pts)

- Utilisez la ligne de commande pour créer un contrôleur pour le projet.

```
amy14@MSI MINGW64 ~/Documents/mesProjets/cadricielWeb/exercice-laravel
• $ php artisan make:controller HomeController
```

5. Définition des routes : (2pts)

- Configurez au moins 4 routes pointant vers le contrôleur créé précédemment.

```
Route::get('/', [HomeController::class, 'index'])->name('home');

Route::get('/about', [HomeController::class, 'about'])->name('about');

Route::get('/contact', [HomeController::class, 'contact'])->name('contact');

Route::post('/contact', [HomeController::class, 'sendContact'])->name('contact.send');
```

6. Utilisation d'un formulaire : (2pts)

- Intégrez un formulaire dans votre site et utilisez la méthode POST pour soumettre les données.

```
@section('content')
<section class="card">
  <h2>Formulaire de contact</h2>

  <form method="POST" action="{{ route('contact.send') }}">
    @csrf

    <label>Nom</label>
    <input type="text" name="name" required>

    <label>Email</label>
    <input type="email" name="email" required>

    <label>Message</label>
    <textarea name="message" required></textarea>

    <button type="submit" class="btn btn-primary">
      Envoyer
    </button>
  </form>
</section>
@endsection
```

7. Traitement des données reçues : (2pts)

- Dans le contrôleur, traitez les données reçues par le formulaire en utilisant la méthode POST, puis affichez-les sur une page.

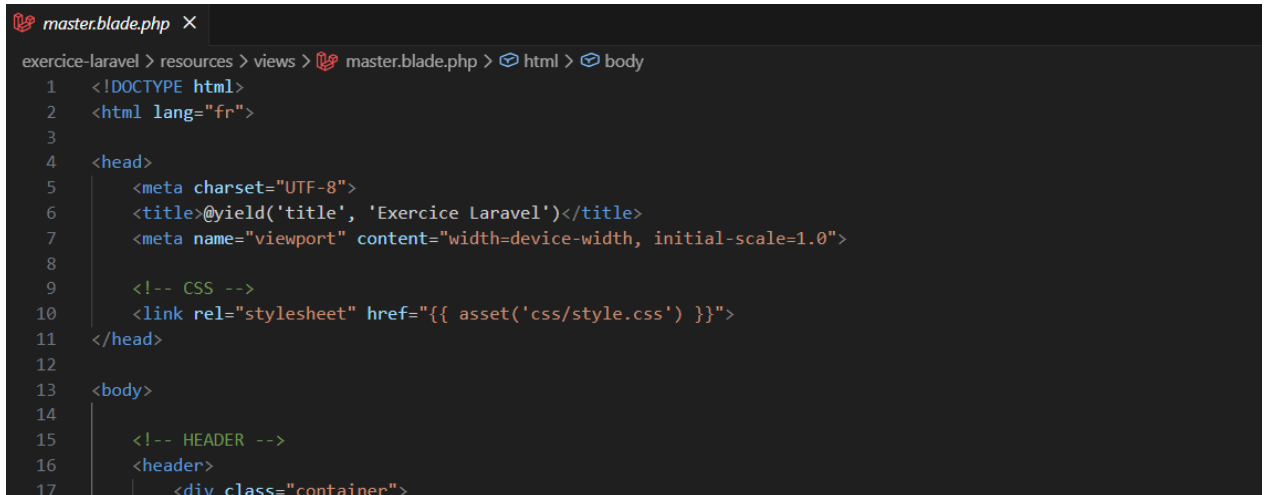
```
public function sendContact(Request $request)
{
    // Validation des données
    $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|email',
        'message' => 'required|string',
    ]);

    // Récupération des données envoyées
    $name = $request->input('name');
    $email = $request->input('email');
    $message = $request->input('message');

    // Envoi des données à la vue
    return view('contact-result', compact('name', 'email', 'message'));
}
```

8. Utilisation de fichiers Blade : (2pts)

- Utilisez un fichier "Master" avec des fichiers Blade pour éviter la répétition de code.



```
exercice-laravel > resources > views > master.blade.php > html > body
1 <!DOCTYPE html>
2 <html lang="fr">
3
4 <head>
5     <meta charset="UTF-8">
6     <title>@yield('title', 'Exercice Laravel')</title>
7     <meta name="viewport" content="width=device-width, initial-scale=1.0">
8
9     <!-- CSS -->
10    <link rel="stylesheet" href="{ asset('css/style.css') }">
11 </head>
12
13 <body>
14
15     <!-- HEADER -->
16     <header>
17         <div class="container">
```

9. Publication sur WebDev : (2pts)

- Utilisez le fichier .htaccess fourni pour publier votre projet sur WebDev. Ajoutez ce fichier à la racine de votre projet. **(Ne pas utiliser webDev)**

10. Publication sur GitHub : (2pts)

- Publiez votre projet sur GitHub en mode public et incluez le lien dans la documentation.
(Lien GitHub ici) https://github.com/marieAndreeHealeyCote/laravel_exercice1

11. Qualité du projet : (2pts)

- Créativité, organisation, documentation, code propre, commentaire, etc.

12. Fourniture de la documentation :

- Enregistrez le projet avec une extension ZIP et ajoutez la documentation dans la racine.
- Fournissez un fichier au format Word/PDF contenant toutes les lignes de commandes utilisées pour les exercices 1, 2 et 3, ainsi que les références, les liens vers WebDev et GitHub, l'absence de ces fichiers entraînera l'attribution d'une note zéro.

13. Soumission du projet :

- Déposez le fichier ZIP sur Moodle